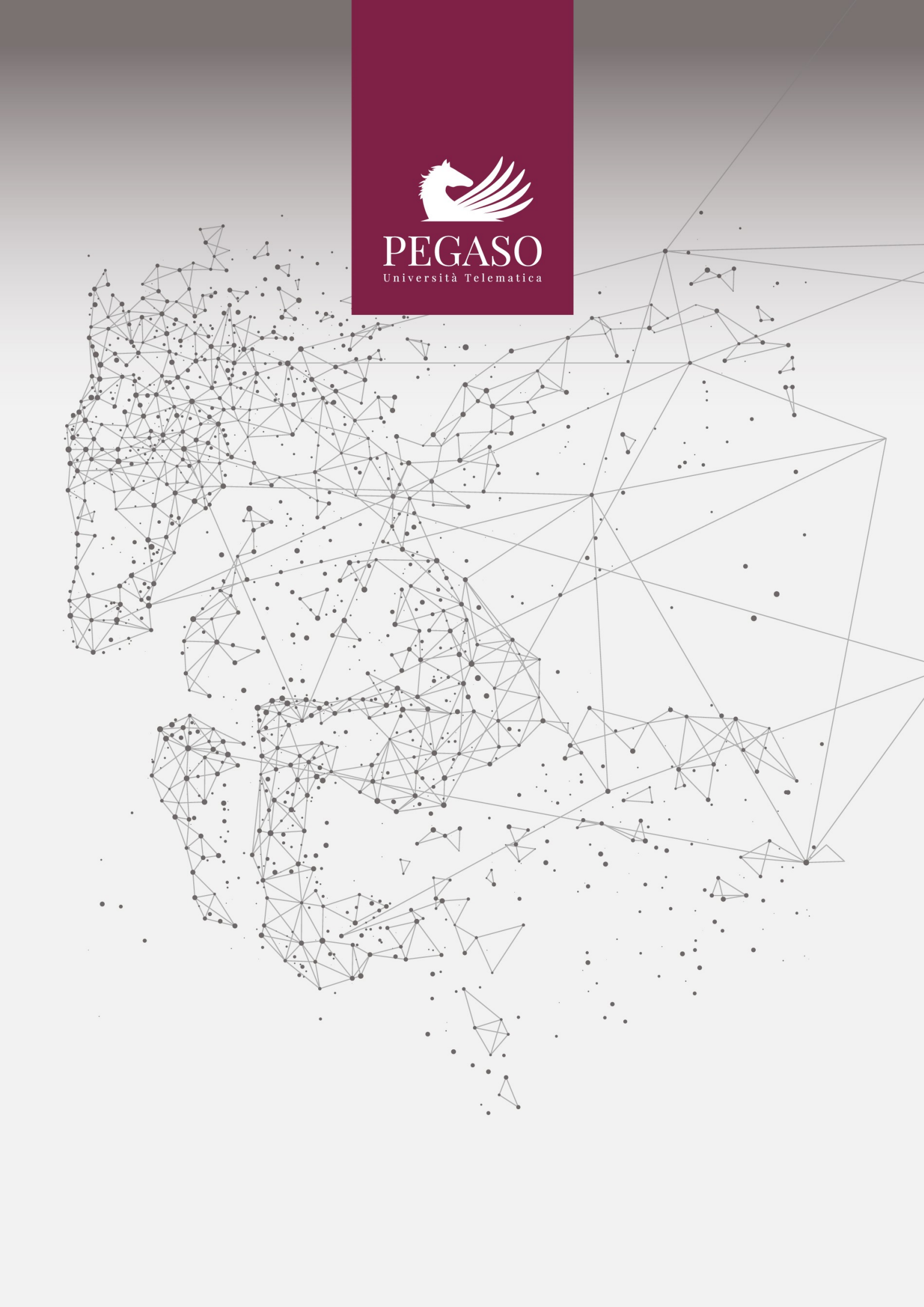

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. TODOLIST 3
2. FORM VALIDATION 6
3. DOVE INSERIRE IL NOSTRO JAVASCRIPT 9
4. PROGETTO – INDOVINA IL NUMERO 11
BIBLIOGRAFIA E SITOGRAFIA 16

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. ToDoList

Iniziamo creando un documento HTML che contenga un titolo, un form con un campo di input di testo e un pulsante di invio per aggiungere nuovi elementi alla lista, e un'area dove mostrare la lista degli elementi aggiunti.

Il form dovrebbe avere un attributo id univoco che possiamo utilizzare per recuperare il form tramite JavaScript, e l'area di visualizzazione dovrebbe avere un attributo id che useremo per recuperare l'elemento dall'HTML tramite JavaScript.

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>TODO List</title>
    <style>
      .selected {
        background-color: #f7d7d7;
      }
    </style>
  </head>
  <body>
    <h1>TODO List</h1>
    <form id="add-item-form">
      <label for="item-input">Aggiungi un nuovo elemento:</label>
      <input type="text" id="item-input" placeholder="Scrivi qui...">
      <button type="submit">Aggiungi</button>
    </form>
    <div id="item-list"></div>
    <button id="remove-selected-button">Rimuovi selezionati</button>
    <script src="script.js"></script>
  </body>
</html>
```

Aggiungiamo un listener sull'evento submit del form per gestire l'aggiunta di un nuovo elemento alla lista: quando l'utente invia il form, la pagina si ricarica, quindi dobbiamo prevenire questo comportamento predefinito utilizzando il metodo **preventDefault()** dell'oggetto evento; in questo modo possiamo gestire l'aggiunta di un nuovo elemento tramite JavaScript senza dover ricaricare la pagina.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
const itemList = document.querySelector('#item-list');

itemList.addEventListener('click', function(event) {
  if (event.target.tagName === 'LI') {
    event.target.classList.toggle('selected');
  }
});

const form = document.querySelector('#add-item-form');
form.addEventListener('submit', function(event) {
  event.preventDefault();
  const input = document.querySelector('#item-input');
  const value = input.value.trim();
  if (value !== '') {
    addItemToList(value);
    input.value = '';
    input.focus();
  }
});

function addItemToList(value) {
  const newItem = document.createElement('li');
  newItem.textContent = value;
  itemList.appendChild(newItem);
}

const removeSelectedButton=document.querySelector('#remove-selected-button');
removeSelectedButton.addEventListener('click', function() {
  const selectedItems = document.querySelectorAll('.selected');
  selectedItems.forEach(function(item) {
    item.remove();
  });
});
```

In questo codice, stiamo selezionando il form tramite l'ID che abbiamo assegnato nell'HTML. Aggiungiamo un listener sull'evento submit del form, in modo da poter gestire l'aggiunta di un nuovo elemento alla lista: quando l'evento si verifica, preveniamo il comportamento predefinito del form utilizzando preventDefault(); quindi recuperiamo il valore dell'input dell'utente rimuovendo eventuali spazi bianchi con il metodo trim(), e verificando che non sia vuoto: se il valore non è vuoto, chiamiamo una funzione addItemToList che si occupa di aggiungere un nuovo elemento alla lista. Questa funzione deve creare un nuovo elemento HTML, aggiungere il testo dell'input come contenuto dell'elemento, e aggiungere l'elemento alla lista degli elementi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Aggiungiamo un listener sull'evento click degli elementi della lista per gestire la loro rimozione: quando l'utente clicca su un elemento della lista dobbiamo rimuoverlo dalla lista; tuttavia, poiché gli elementi della lista possono essere generati dinamicamente, dobbiamo utilizzare la delega degli eventi per gestire l'evento click sull'intero elenco degli elementi.

Sull'itemList aggiungiamo un listener sull'evento click: quando l'evento si verifica, controlliamo se l'elemento cliccato è un elemento li; se lo è, stiamo rimuovendo l'elemento dalla lista utilizzando il metodo remove(). Aggiungiamo inoltre un effetto visivo sull'elemento selezionato: quando l'utente clicca su un elemento della lista, potrebbe essere utile evidenziare l'elemento selezionato per migliorare l'esperienza dell'utente.

Aggiungiamo inoltre un pulsante in modo che l'utente possa rimuovere rapidamente tutti gli elementi selezionati: aggiungiamo un listener sull'evento click del pulsante "Rimuovi selezionati" per gestire la rimozione degli elementi selezionati in modo tale che quando l'utente clicca sul pulsante, selezioniamo tutti gli elementi li che hanno la classe selected, e li rimuoviamo dalla lista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Form Validation

La validazione di un form in JavaScript è un'operazione comune, che può essere eseguita per verificare se i dati inseriti dall'utente sono validi e conformi ai requisiti specificati.

Iniziamo creando una funzione JavaScript che verrà chiamata quando viene inviato il form. Per fare ciò, è possibile utilizzare l'attributo **onsubmit** dell'elemento form:

```
<form onsubmit="return validaForm();">
  ...
</form>
```

In questo esempio, stiamo utilizzando l'attributo onsubmit per chiamare la funzione **validaForm()** quando viene inviato il form. All'interno della funzione **validaForm()**, è possibile accedere ai valori dei campi del form utilizzando le proprietà value degli elementi HTML.

```
function validaForm() {
  var nome = document.getElementById("nome").value;
  var email = document.getElementById("email").value;
  ...
}
```

In questo esempio, stiamo ottenendo i valori dei campi nome e email utilizzando il metodo **getElementById()** e la proprietà **value**.

Una volta che hai ottenuto i valori dei campi del form, puoi eseguire la validazione utilizzando le condizioni if.

```
function validaForm() {
  var nome = document.getElementById("nome").value;
  var email = document.getElementById("email").value;
  if (nome == "") {
    alert("Inserisci il tuo nome");
    return false;
  }
  if (email == "") {
    alert("Inserisci il tuo indirizzo email");
    return false;
  }
  ...
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, stiamo verificando se i campi nome e email sono vuoti utilizzando una condizione if. Se uno dei campi è vuoto, viene visualizzato un messaggio di errore e viene restituito false per impedire l'invio del form.

Possiamo anche utilizzare le espressioni regolari per eseguire una validazione più avanzata sui campi del form. Ad esempio, per verificare se l'indirizzo email inserito è valido, puoi utilizzare la seguente espressione regolare:

```
function validaForm() {
  var email = document.getElementById("email").value;
  var regExpEmail=/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/;
  if (!regExpEmail.test(email)) {
    alert("Inserisci un indirizzo email valido");
    return false;
  }
  ...
}
```

In questo esempio, stiamo utilizzando l'espressione regolare /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/ per verificare se l'indirizzo email inserito è valido. Se l'indirizzo email non corrisponde al formato specificato dall'espressione regolare, viene visualizzato un messaggio di errore e viene restituito false.Infine, se tutti i campi del form sono validi, possiamo restituire true dalla funzione validaForm() per permettere l'invio del form.

```
function validaForm() {
  var nome = document.getElementById("nome").value;
  var email = document.getElementById("email").value;
  ...

  if (nome == "") {
    alert("Inserisci il tuo nome");
    return false;
  }

  if (email == "") {
    alert("Inserisci il tuo indirizzo email");
    return false;
  }

  ...

  return true;
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, stiamo restituendo true dalla funzione validaForm() se tutti i campi del form sono validi. In caso contrario, viene restituito false e viene visualizzato un messaggio di errore.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Dove inserire il nostro Javascript

Consideriamo il seguente codice:

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>TEST</title>
</head>
<body>
  <p id="myP">Hello world</p>
</body>
</html>
```

Consideriamo il seguente script contenuto in un file esterno script.js

```
alert(document.getElementById("myP").innerHTML)
console.log("test")
```

A questo punto, abbiamo 3 possibilità di inserire tale script nella pagina:

1. Nell'head (posizione 1)
2. Nel body subito dopo il tag body (posizione 2)
3. In un punto qualunque del body (posizione 3)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width">
  <title>TEST</title>
  <!-- POSIZIONE 1 -->
</head>
<body>
  <!-- POSIZIONE 2 -->
  <p id="myP">Hello world</p>
  <!-- POSIZIONE 3 -->
</body>
</html>
```

In posizione 1 e 2, l'alert presente nel Javascript non ha effetto, questo perché:

1. In posizione 1 lo script viene caricato e quindi eseguito prima che l'intero contenuto della pagina HTML sia stato caricato; ciò significa che se lo script dipende da elementi specifici della pagina

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

HTML (come nel nostro caso), potrebbe non funzionare correttamente se questi elementi non sono ancora stati caricati (ed infatti l'alert non ha effetto); in tal caso il console.log determina:

Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')

questo perché l'elemento <p> della pagina non è stato ancora caricato

2. In posizione 2 lo script viene caricato e quindi eseguito subito appena il body viene costruito, ma prima che l'intero contenuto della pagina HTML sia stato caricato; ciò significa che se lo script dipende da elementi specifici della pagina HTML (come nel nostro caso) che si trovano DOPO il caricamento dello script, lo script potrebbe non funzionare correttamente (ed infatti l'alert non ha effetto); in tal caso il console.log determina:

   Uncaught TypeError: Cannot read properties of null (reading 'innerHTML')

   questo perché l'elemento <p> della pagina non è stato ancora caricato

In posizione 3 invece tutto procede correttamente perché tutti gli elementi della pagina sono stati caricati già e quindi lo script può avere effetto: l'alert è visualizzato ed il console.log determina la scrittura di test.

In conclusione:

- Se lo script si trova nell'head, questo viene invocato immediatamente; se questo dipende da elementi specifici della pagina HTML lo script potrebbe non funzionare correttamente
- Se lo script si trova nel body, questo viene invocato immediatamente durante il caricamento della pagina rispettando il suo posizionamento; se lo script dipende da elementi specifici della pagina HTML che si trovano DOPO (a livello di posizionamento) il caricamento dello script, lo script potrebbe non funzionare correttamente
- Se lo script si trova nel body subito prima la chiusura del tag stesso, siamo certi che tutti gli elementi della pagina sono stati caricati, pertanto se lo script dipende da elementi specifici della pagina HTM, siamo certi che lo script funzionerà correttamente

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Progetto – indovina il numero

Il codice di seguito rappresenta un semplice gioco "Indovina il numero" che mette alla prova le abilità di logica e di conteggio dell'utente.

Il codice JavaScript è la parte interattiva della pagina che gestisce la logica del gioco "Indovina il numero". In particolare, genera un numero casuale tra 1 e 100, tiene traccia del numero di tentativi fatti dall'utente, imposta il numero massimo di tentativi e il tempo massimo per indovinare il numero, avvia il timer e definisce una funzione per verificare il tentativo dell'utente.

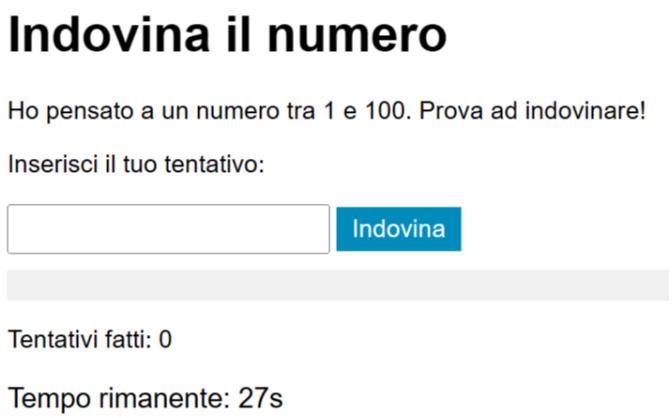


Figura 1 - Indovina il numero

La funzione "verificaTentativo" gestisce la validità dell'input dell'utente, controlla se il numero inserito è corretto e restituisce un messaggio di errore se l'utente ha inserito un valore non valido. Se il numero inserito dall'utente è corretto, la funzione interrompe il timer, restituisce un messaggio di successo e disabilita l'input di testo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

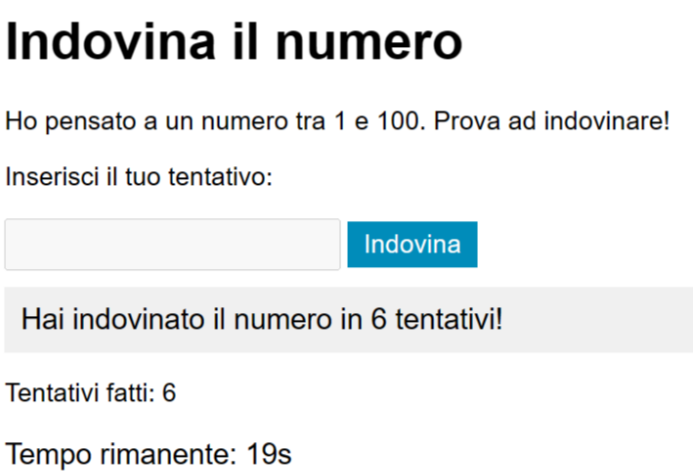


Figura 2 - Numero indovinato

In caso contrario, la funzione fornisce un suggerimento all'utente sul numero corretto da indovinare e tiene traccia del numero di tentativi effettuati. Se l'utente ha esaurito tutti i tentativi, la funzione interrompe il timer e disabilita l'input di testo.

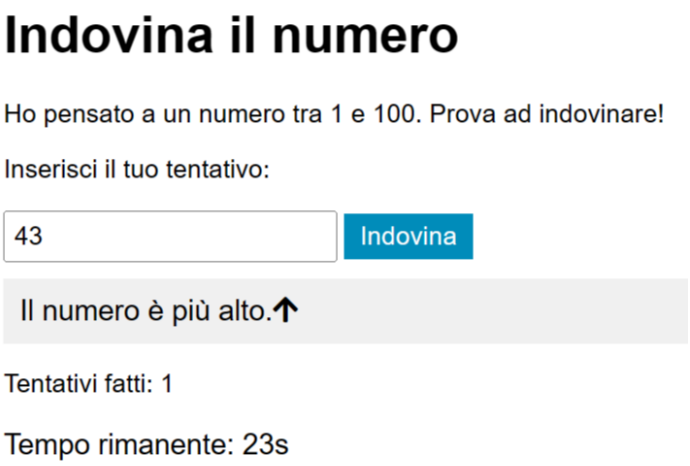


Figura 3 - Numero errato

Infine, la funzione "aggiornaTimer" aggiorna il valore del timer ogni secondo e controlla se il tempo massimo per indovinare il numero è scaduto. Se il tempo è scaduto, la funzione interrompe il timer e disabilita l'input di testo.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Di seguito il codice:

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>Indovina il numero</title>
    <link rel="stylesheet" href="style.css">
    <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
awesome/5.15.3/css/all.min.css" integrity="sha512-
iBBXm8fW90+nuLcSKlbmrPcLa0OT92xO1BIsZ+ywDWZCvqsWgccV3gFoRBv0z+8dLJgyAHIhR35VZc2oM/gI1w=
=" crossorigin="anonymous" referrerpolicy="no-referrer" />
  </head>
  <body>
    <h1>Indovina il numero</h1>
    <p>Ho pensato a un numero tra 1 e 100. Prova ad indovinare!</p>
    <p>Inserisci il tuo tentativo:</p>
    <input type="text" id="tentativo">
    <input type="button" value="Indovina" onclick="verificaTentativo()">
    <div class="risultato" id="risultato"></div>
    <p>Tentativi fatti: <span id="num-tentativi">0</span></p>
    <p class="timer">Tempo rimanente: <span id="timer"></span></p>
    <script src="script.js"></script>
  </body>
</html>
```

CSS

```
body {
  font-family: Arial, sans-serif;
}

input[type="text"] {
  padding: 5px;
  font-size: 16px;
}

input[type="button"] {
  padding: 5px 10px;
  font-size: 16px;
  background-color: #008CBA;
  border: none;
  color: #FFF;
  cursor: pointer;
}

.risultato {
  margin-top: 10px;
  padding: 10px;
  background-color: #F0F0F0;
  font-size: 18px;
}
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
.timer {
  font-size: 18px;
}
```

Javascript

```
// Generazione del numero casuale tra 1 e 100
var numeroMisterioso = Math.floor(Math.random() * 100) + 1;

// Contatore dei tentativi fatti
var numTentativi = 0;

// Numero massimo di tentativi consentiti
var maxTentativi = 10;

// Tempo massimo per indovinare il numero in secondi
var tempoMassimo = 30;

// Tempo rimanente
var tempoRimanente = tempoMassimo;

// Intervallo del timer
var timerInterval;

// Recupero dell'input dell'utente
var inputTentativo = document.getElementById("tentativo");

// Aggiunta dell'evento "keypress" sull'input
inputTentativo.addEventListener("keypress", function(event) {
  // Verifica se il tasto premuto è il tasto "Invio"
  if (event.keyCode === 13) {
    verificaTentativo();
  }
});

// Funzione per verificare il tentativo
function verificaTentativo() {
  // Incremento del numero di tentativi fatti
  numTentativi++;

  // Recupero del valore inserito dall'utente
  var valoreTentativo = parseInt(inputTentativo.value);

  // Verifica se il valore inserito è un numero valido
  if (isNaN(valoreTentativo)) {
    document.getElementById("risultato").innerHTML = "Devi inserire un numero!";
  } else if (valoreTentativo < 1 || valoreTentativo > 100) {
    document.getElementById("risultato").innerHTML = "Il numero deve essere compreso
tra 1 e 100!";
  } else {
    // Verifica se il numero inserito è corretto
    if (valoreTentativo === numeroMisterioso) {
      clearInterval(timerInterval);
      document.getElementById("risultato").innerHTML = "Hai indovinato il numero in " +
numTentativi + " tentativi!";
      inputTentativo.disabled = true;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
    } else {
      var messaggioRisultato = "Il numero è ";
      var iconaRisultato = "";
      if (valoreTentativo < numeroMisterioso) {
        messaggioRisultato += "più alto.";
        iconaRisultato = '<i class="fas fa-arrow-up"></i>';
      } else {
        messaggioRisultato += "più basso.";
        iconaRisultato = '<i class="fas fa-arrow-down"></i>';
      }
      document.getElementById("risultato").innerHTML = messaggioRisultato +
iconaRisultato;
    }

    // Verifica se il numero massimo di tentativi è stato raggiunto
    if (numTentativi >= maxTentativi) {
      clearInterval(timerInterval);
      document.getElementById("risultato").innerHTML = "Hai esaurito i tentativi!";
      inputTentativo.disabled = true;
    }
  }

  // Aggiornamento del numero di tentativi effettuati
  var numTentativiElement = document.getElementById("num-tentativi");
  numTentativiElement.innerHTML = numTentativi;

  // Azzeramento del valore dell'input
  inputTentativo.value = "";
}

// Funzione per aggiornare il timer
function aggiornaTimer() {
  tempoRimanente--;
  if (tempoRimanente <= 0) {
    clearInterval(timerInterval);
    document.getElementById("risultato").innerHTML = "Tempo scaduto!";
    document.getElementById("tentativo").disabled = true;
  } else {
    document.getElementById("timer").innerHTML = tempoRimanente + "s";
  }
}

// Avvio del timer
timerInterval = setInterval(aggiornaTimer, 1000);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia e sitografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. *Introduzione alle Tecnologie Web*, McGraw-Hill, 2010
- A. S. Tanembaum. *Computer Networks* (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. *Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR*, 2005
- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).